ESP32 MODQ MODBUS GATEWAY

Industrial RS485 Modbus RTU to Web UI and MQTT Broker Gateway

User & Technical Reference Manual

Device Metadata:

Hardware: ESP32 ModQ Development Board
Firmware Version: v1.0.0 (Multilingual Edition)
Protocols: Modbus RTU (RS485) & MQTT (over WiFi)
Web UI IP Default: 192.168.4.1 (AP Mode) / Port 80

1. Introduction & Specifications

The ESP32 ModQ Modbus Gateway is an industrial-grade, versatile IoT gateway designed to aggregate telemetry from multiple Modbus RTU slave devices over a half-duplex RS485 bus, provide local data visualization via an offline-capable Web dashboard, and publish real-time data to MQTT brokers.

Key system parameters and specifications:

- **Multi-WiFi Connection:** Saves up to 3 WiFi network credentials and automatically transitions between them if connection is lost.
- **Fallback Access Point:** Starts a local, open WiFi network "ModQ_AP" at IP 192.168.4.1 so the dashboard is always accessible.
- **Multilingual Interface:** Saves user language choice and localizes all elements into English (default), Vietnamese, Spanish, or Portuguese.
- **Bi-directional MQTT:** Supports publishing JSON telemetry per device and subscribing to remote command payloads to write registers.
- **Visual LEDs Feedback:** LED1 flashes on WiFi/Web activity; LED2 flashes on Modbus transactions.

2. Hardware Interface & Pinout

The gateway utilizes the hardware peripherals on the ESP32 ModQ board. Specifically, it employs the second hardware UART port (UART2) connected to an onboard SP485EE RS485 transceiver. Transmit/Receive flow direction control (DE/RE) is managed automatically by the hardware circuit.

Component / Signal	ESP32 GPIO Pin	Description
RS485 RX	GPIO 16 (RX2)	Receives data packets from the Modbus RS485 bus.
RS485 TX	GPIO 17 (TX2)	Transmits query packets to the Modbus RS485 bus.
DE / RE Flow Control	Hardware managed	SP485EE chip auto-switches TX/RX states.
LED 1 (SYS Status)	GPIO 25	Flashes during HTTP Web activity and network status updates.
LED 2 (MODBUS Query)	GPIO 27	Flashes briefly during Modbus RTU read/write transactions.

3. Installation & Compilation

The firmware is written as an Arduino sketch. To compile and upload it onto the ESP32 board, follow the steps below:

- **Libraries Needed:** Install "ArduinoJson" (v6.x or v7.x) and "PubSubClient" (by Nick O'Leary) via the Library Manager.
- **IDE Configuration:** Open Canopus_ModbusApp.ino in Arduino IDE. Go to Tools > Board and select "ESP32 Dev Module".
- **Wiring Details:** Connect the USB Type-C port of the ModQ board to your computer and select the correct COM port.
- **Firmware Upload:** Click the "Upload" button. The board LEDs will flash three times on successful boot-up.

4. Initial COM Port Configuration (Serial CLI)

For brand new boards or situations where WiFi settings are missing, you can plug in the USB Type-C cable and configure the gateway over the serial interface using simple text commands. Open a Serial Monitor at 115200 baud.

Available Command Reference:

- **help:** Displays the Serial Command reference help menu.
- **status:** Prints IP addresses, WiFi/MQTT connection state, uptime, and free RAM.
- **config:** Prints the entire active gateway settings configuration structure in JSON.
- **config set <json>:** Allows setting the complete configuration by pasting a JSON string.
- **wifi add <ssid> <pass>:** Saves a WiFi network configuration (supports up to 3 slots).
- **wifi clear:** Clears all configured WiFi network credentials from flash.
- **mqtt set <srv> <port> [u] [p]:** Sets MQTT broker server address, port, and optional user/pass.
- **restart / reset:** Restarts the gateway or performs a factory reset back to defaults.

Example WiFi Serial Configuration:

```
Received command: wifi add MyHomeWiFi SecretPassword123
SUCCESS: Added WiFi SSID: MyHomeWiFi
Configuring Network Interfaces...
Access Point 'ModQ_AP' started. IP: 192.168.4.1
Registered 1 WiFi networks.
 - Added network SSID: MyHomeWiFi
```

5. Web UI Interface (Multilingual)

The gateway hosts an offline-capable Single-Page Web application on port 80. Connect your device to the "ModQ_AP" WiFi network and navigate to http://192.168.4.1 on your browser.

- **Language Toggle:** Select English, Vietnamese, Spanish, or Portuguese from the top dropdown in the sidebar to translate the dashboard dynamically.
- **WiFi Setup:** Under WiFi Networks, scan nearby APs, select your network, and enter the password. Multi-WiFi operates in the background.
- **Modbus Configuration:** Configure up to 8 Modbus slave devices. Click "Add Device", assign Slave ID, Baudrate, and Polling Interval.
- **Registers Addition:** Define up to 16 registers per device. Supported Datatypes include UINT16, INT16, UINT32, INT32, and FLOAT.

6. MQTT API Specification

The gateway supports two-way communication using MQTT over WiFi. The default topics can be adjusted on the Web UI.

6.1 Telemetry JSON Payload Format (Publish):

The gateway publishes Modbus values to the topic "modq/device/data/<slave_id>" every query interval:

```
{
  "name": "Industrial Sensor 1",
  "sid": 1,
  "online": true,
  "error": 0,
  "regs": {
    "Temperature": 24.85,
    "Humidity": 55.40
  }
}
```

6.2 Control Commands (Subscribe):

Write values to slave registers by publishing JSON payloads to "modq/device/cmd":

```
# Publish to topic: modq/device/cmd
{
  "sid": 1,
  "addr": 100,
  "val": 1250
}
```

7. HTTP REST API Reference

Developers can integrate the gateway with third-party dashboards or SCADA systems using HTTP REST endpoints.

Method	Endpoint	Response	Description
GET	/	HTML	Serves the main dashboard Single Page Web UI.
GET	/api/config	JSON	Returns WiFi, MQTT, and device registers setup.
POST	/api/config	JSON	Submits and applies new configurations.
GET	/api/data	JSON	Returns current polled Modbus values.
GET	/api/status	JSON	Returns WiFi/MQTT connection state, IP, RAM, uptime.
GET	/api/wifi-scan	JSON	Triggers scan of surrounding WiFi APs.
POST	/api/restart	JSON	Reboots the ESP32 gateway module.
POST	/api/reset	JSON	Wipes all flash settings and restarts gateway.